Name of the Student	P Surendra Kumar Reddy
Reg. No	192425224
GitHub Link	https://github.com/suri307/MLA0403-DEEP-LEARNING

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**

COURSE OUTCOME COVERED

CO 2: Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches.(BTL-4)

CO Weightage : 15%

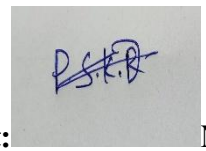
Assessment Tool Weightage: 40%

Duration: 90 minutes

Marks: 25

Code of Conduct:

I P Surendra Kumar Reddy (192425224) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:  **Name of the Student: P Surendra Kumar Reddy**

Scenario-Based Questions

CO-AT2

QUESTIONS: 05 **Total Marks:25**

1. **Unsupervised Training of Neural Networks, Auto Encoders**
   A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

2. **Auto Encoders, Representation Learning**
   A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

3. **Width and Depth of Neural Networks, Activation Functions**
   A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

4. **Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks**
   A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

5. **Deep Learning Applications, Representation Learning**

   A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and	Shows correct relationships	Relationships are weak,	No meaningful

		accurate relationships between concepts	with minor gaps	unclear, or partially incorrect	relationships established
Organization	20%	Concepts are wellstructured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

1. Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

1. Introduction

Electrocardiogram (ECG) signals are widely used to monitor heart activity and diagnose cardiovascular diseases. Hospitals generate thousands of ECG recordings every day, but manually labeling each record is time-consuming, expensive, and requires expert cardiologists. Since the dataset is largely unlabeled, an **Autoencoder**, an unsupervised deep learning model, is an effective solution for learning normal ECG patterns and detecting abnormal signals automatically.

2. Problem Statement

The hospital needs to:

- Analyze thousands of unlabeled ECG signals.
- Detect abnormal heart patterns automatically.
- Reduce dependency on manual diagnosis.
- Improve early disease detection.
- Minimize false alarms while maintaining high accuracy.

3. Recommended Deep Learning Technique

Autoencoder (Unsupervised Neural Network)

An Autoencoder is a neural network trained to reconstruct its input. During training, it learns compressed representations (latent features) of normal ECG signals.

When abnormal ECG signals are given as input, the reconstruction error becomes significantly higher because the model has not learned those abnormal patterns. Therefore, signals with high reconstruction error are classified as anomalies.

4. Justification

The Autoencoder is the most suitable technique because:

- Works without labeled training data.
- Learns hidden characteristics of normal ECG signals.
- Detects unknown cardiac abnormalities.
- Reduces manual effort.
- Produces meaningful feature representations.
- Handles large medical datasets efficiently.
- Supports real-time anomaly detection.

5. Workflow

Step 1: ECG Data Collection

Thousands of ECG recordings are collected from hospital monitoring systems.

↓

Step 2: Data Preprocessing

- Remove noise
- Normalize signal values
- Remove missing samples

- Segment ECG beats

↓

Step 3: Autoencoder Training

Only normal ECG patterns are used.

The Autoencoder learns compressed representations through:

- Encoder
- Latent Space
- Decoder

↓

Step 4: Reconstruction

The trained model reconstructs each ECG signal.

↓

Step 5: Error Calculation

Calculate reconstruction error using:

- Mean Squared Error (MSE)

↓

Step 6: Anomaly Detection

If

Reconstruction Error > Threshold

↓

ECG is Abnormal

Else

ECG is Normal

6. Architecture Diagram

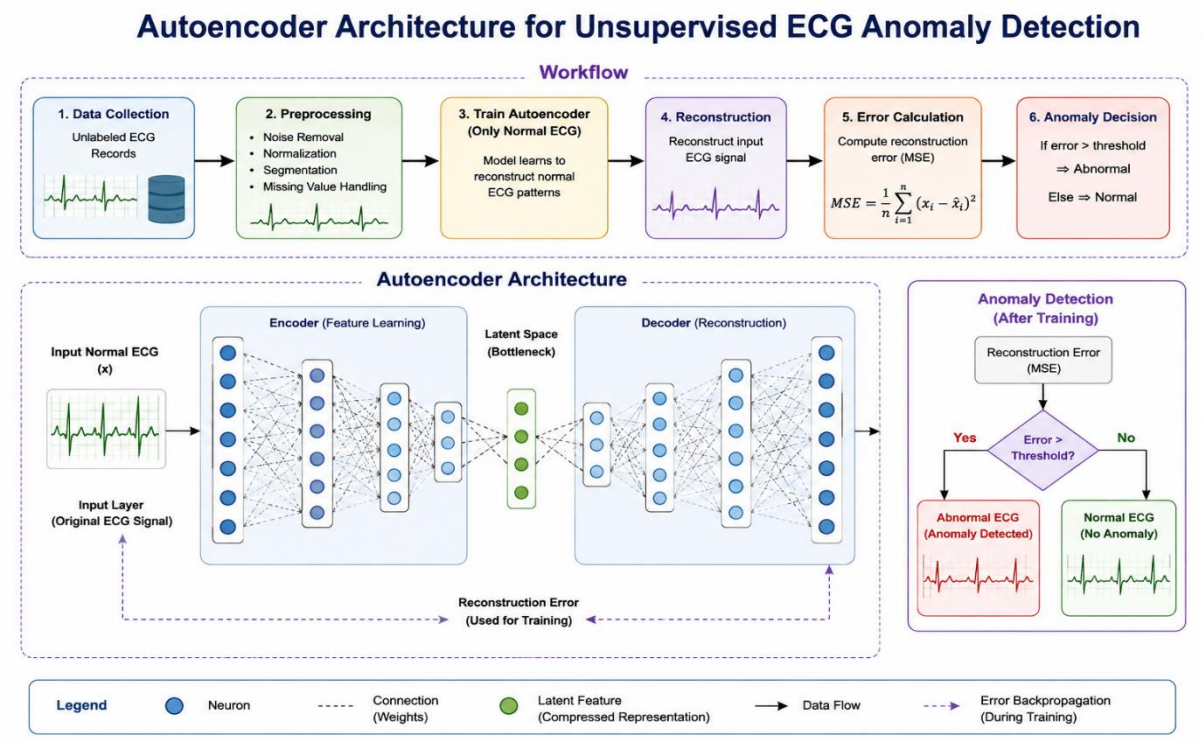


Figure 1: Autoencoder Architecture for ECG Anomaly Detection

7. Advantages

- No labeled dataset required.
- Learns meaningful ECG features automatically.
- Detects rare heart abnormalities.
- Reduces workload for cardiologists.

- High accuracy for anomaly detection.
- Scalable for large hospital datasets.
- Supports continuous patient monitoring.

8. Limitations

- Requires sufficient normal ECG data for training.
- Threshold selection affects detection accuracy.
- Performance decreases if training data contains abnormal signals.
- Complex abnormalities may require deeper architectures.

9. Real-World Applications

- Cardiac disease diagnosis
- Intensive Care Unit (ICU) monitoring
- Wearable health devices
- Remote patient monitoring
- Smart hospital systems
- Telemedicine platforms

10. Conclusion

An Autoencoder is an effective unsupervised deep learning technique for detecting abnormal ECG patterns in unlabeled medical datasets. By learning the characteristics of normal ECG signals and identifying reconstruction errors, it accurately detects anomalies without requiring

manual labeling. This approach improves early diagnosis, reduces healthcare costs, and enhances patient monitoring in modern hospitals.

11. IEEE References

1. G. E. Hinton and R. R. Salakhutdinov, "Reducing the Dimensionality of Data with Neural Networks," *Science*, vol. 313, no. 5786, pp. 504–507.

2. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press.

3. D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," *International Conference on Learning Representations (ICLR)*.

4. J. Schmidhuber, "Deep Learning in Neural Networks: An Overview," *Neural Networks*, vol. 61, pp. 85–117.

2.Auto Encoders, Representation Learning
A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

1. Introduction

Retail companies collect large volumes of customer purchase data, including product categories, purchase frequency, transaction amounts, shopping preferences, and payment methods. This high-dimensional data often contains redundant and correlated features, making customer segmentation computationally expensive and less accurate. An **Autoencoder** is an unsupervised deep learning model that learns compact feature representations (latent features) while preserving essential information. These compressed features improve the efficiency and accuracy of customer segmentation.

2. Problem Statement

The retail company aims to:

- Reduce the dimensionality of customer purchase data.
- Remove redundant and irrelevant features.
- Preserve meaningful purchasing patterns.
- Improve customer segmentation accuracy.
- Reduce computational complexity.
- Support personalized marketing strategies.

3. Recommended Deep Learning Technique

Autoencoder for Dimensionality Reduction

An Autoencoder consists of two main components:

- **Encoder:** Compresses the high-dimensional customer data into a lower-dimensional latent representation.

- **Decoder:** Reconstructs the original data from the compressed representation.

The compressed features obtained from the latent space are then used as input for clustering algorithms (such as K-Means) to group customers with similar purchasing behavior.

4. Justification

An Autoencoder is suitable because it:

- Learns feature representations without labeled data.

- Removes redundant and noisy information.

- Preserves important purchasing patterns.

- Reduces storage and computation requirements.

- Improves clustering and customer segmentation performance.

- Enables efficient analysis of large retail datasets.

5. Workflow

Step 1: Customer Purchase Data Collection

Collect customer transaction records from retail databases.

↓

Step 2: Data Preprocessing

- Remove missing values

- Encode categorical attributes

- Normalize numerical features
- Remove duplicate records

↓

Step 3: Autoencoder Training

The encoder compresses customer purchase data into a latent feature representation, while the decoder reconstructs the original data.

↓

Step 4: Feature Extraction

Extract compressed features from the latent space.

↓

Step 5: Customer Segmentation

Use the compressed features as input to clustering algorithms (e.g., K-Means) to group customers based on purchasing behavior.

↓

Step 6: Business Insights

Analyze customer groups to support targeted marketing, product recommendations, and customer relationship management.

6. Architecture Diagram

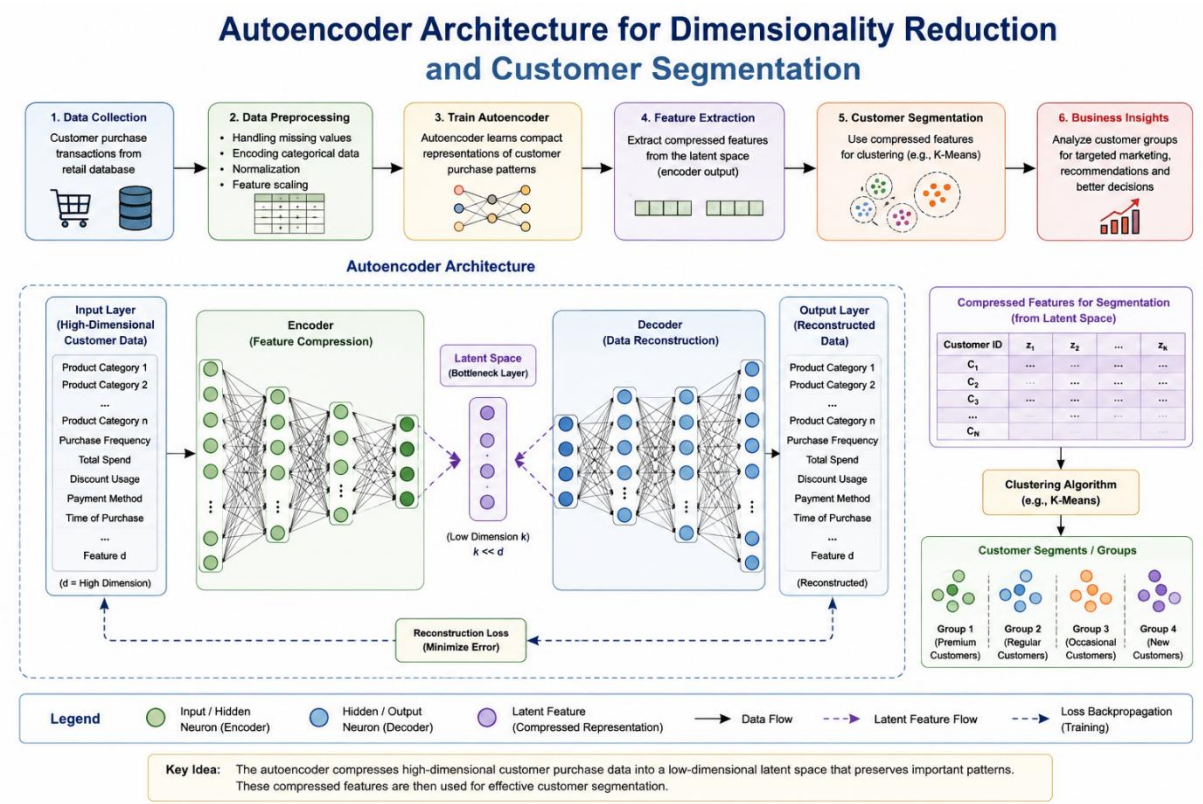


Figure 2: Autoencoder Architecture for Customer Purchase Data Dimensionality Reduction and Customer Segmentation

7. Advantages

- Reduces data dimensionality effectively.
- Removes redundant and noisy features.
- Improves clustering accuracy.
- Reduces computational cost.
- Enhances customer segmentation quality.
- Supports personalized marketing and recommendation systems.
- Scales efficiently to large retail datasets.

8. Limitations

- Requires sufficient training data.
- Selecting the appropriate latent space dimension is challenging.
- Training deep autoencoders can be computationally intensive.
- Reconstruction quality depends on model architecture.

9. Real-World Applications

- Customer segmentation
- Product recommendation systems
- Market basket analysis
- Customer behavior prediction
- Personalized promotions
- Retail business intelligence

10. Conclusion

An Autoencoder is an effective deep learning technique for reducing the dimensionality of customer purchase data. By learning compact feature representations, it removes redundant information while preserving meaningful purchasing patterns. These compressed features enable more accurate customer segmentation, helping retail companies improve marketing strategies, customer relationship management, and business decision-making.

11. IEEE References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press.

2. G. E. Hinton and R. R. Salakhutdinov, "Reducing the Dimensionality of Data with Neural Networks," *Science*, vol. 313, no. 5786, pp. 504–507.

3. D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," *International Conference on Learning Representations (ICLR)*.

4. J. Schmidhuber, "Deep Learning in Neural Networks: An Overview," *Neural Networks*, vol. 61, pp. 85–117.

3.Width and Depth of Neural Networks, Activation Functions
A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

1. Introduction

Facial recognition systems are widely used in security, surveillance, attendance management, and smartphone authentication. A shallow neural network contains only a few hidden layers and has limited capability to learn complex facial features such as eyes, nose, facial expressions, lighting variations, and head poses. To improve recognition performance, a **Deep Neural Network (DNN)** with appropriate activation functions should be used.

2. Problem Statement

The facial recognition system currently suffers from:

- Low recognition accuracy.
- Poor feature extraction capability.
- Difficulty recognizing faces under different lighting conditions.
- Overfitting or underfitting due to limited network depth.
- Inability to learn complex facial representations.

3. Recommended Solution

Deep Neural Network (DNN) with ReLU Activation Function

To improve performance:

- Increase the number of hidden layers (depth).
- Increase the number of neurons (width) in important layers.
- Use **ReLU** activation in hidden layers.

- Use **Softmax** activation in the output layer for face classification.
- Apply **Batch Normalization** and **Dropout** to improve learning and reduce overfitting.

4. Justification

A deeper neural network provides:

- Better hierarchical feature extraction.
- Improved learning of complex facial patterns.
- Higher recognition accuracy.
- Better generalization.
- Reduced vanishing gradient problem when using ReLU.
- Faster convergence during training.

5. Workflow

Step 1: Face Image Collection

Collect facial images from cameras or image datasets.

↓

Step 2: Image Preprocessing

- Resize images
- Normalize pixel values
- Remove noise
- Face alignment

↓

Step 3: Deep Neural Network

Input Layer

↓

Multiple Hidden Layers

↓

ReLU Activation

↓

Batch Normalization

↓

Dropout Layer

↓

Output Layer (Softmax)

↓

Step 4: Face Classification

Identify the person's identity.

↓

Step 5: Performance Evaluation

Measure:

- Accuracy
- Precision
- Recall
- F1-score

6. Architecture Diagram

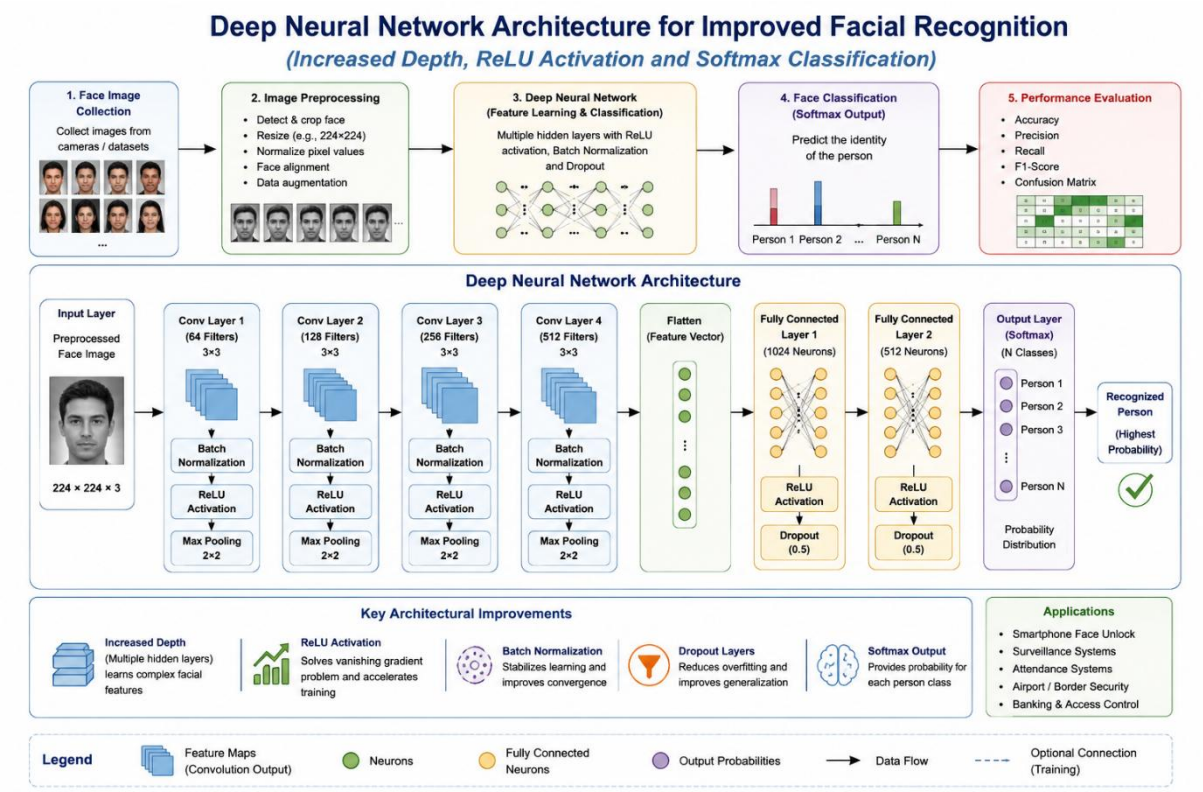


Figure 3: Deep Neural Network Architecture with Increased Depth, ReLU Activation, Batch Normalization, and Softmax Classification for Facial Recognition

7. Advantages

- Learns complex facial features.
- Higher recognition accuracy.
- Faster convergence using ReLU.

- Reduces overfitting through Dropout.
- Stable learning with Batch Normalization.
- Works well with large image datasets.

8. Limitations

- Requires high computational resources.
- Needs large labeled datasets.
- Longer training time.
- Performance depends on image quality.

9. Real-World Applications

- Smartphone Face Unlock
- Airport Security
- Attendance Systems
- Banking Authentication
- Smart Surveillance
- Border Control

10. Conclusion

Increasing the depth and width of a neural network significantly improves facial recognition performance by enabling the model to learn complex facial features. The use of **ReLU activation**, **Batch Normalization**, **Dropout**, and **Softmax classification** results in higher accuracy, faster convergence, and better generalization, making the system suitable for real-world facial recognition applications.

11. IEEE References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press.

2. Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444.

3. A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *NeurIPS*, 2012.

4. K. He et al., "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," *ICCV*, 2015.

4.Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks
A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

1. Introduction

Streaming platforms such as Netflix and Disney+ generate massive amounts of user viewing data every day. Since most of this data is unlabeled, traditional supervised learning methods are not ideal. A **Restricted Boltzmann Machine (RBM)** is an unsupervised neural network that learns hidden patterns from user–movie interactions and predicts movies that users are likely to enjoy. By discovering latent preferences, RBMs provide accurate and personalized movie recommendations.

2. Problem Statement

The streaming service aims to:

- Analyze users' movie viewing history.
- Learn user preferences without labeled data.
- Recommend relevant movies automatically.
- Improve recommendation accuracy.
- Increase user engagement and satisfaction.

3. Recommended Deep Learning Technique

Restricted Boltzmann Machine (RBM)

A Restricted Boltzmann Machine is a two-layer neural network consisting of:

- **Visible Layer:** Represents users' movie ratings or viewing history.

- **Hidden Layer:** Learns latent features such as genre preferences, favorite actors, and viewing habits.

The RBM learns the relationship between users and movies during training and predicts unseen movies that match a user's interests.

4. Justification

RBMs are suitable because they:

- Learn hidden user preferences automatically.
- Require no labeled training data.
- Capture complex relationships between users and movies.
- Handle sparse user–movie rating matrices effectively.
- Improve personalized recommendation quality.
- Scale well to large streaming platforms.

5. Workflow

Step 1: Collect User Viewing History

Gather user watch history, ratings, and interactions.

↓

Step 2: Data Preprocessing

- Remove missing values
- Normalize ratings

- Create User–Movie Interaction Matrix

↓

Step 3: RBM Training

Train the RBM using unsupervised learning to discover hidden user preferences.

↓

Step 4: Hidden Feature Learning

The hidden layer learns:

- Favorite genres
- Preferred actors
- Viewing patterns
- Movie interests

↓

Step 5: Recommendation Generation

Predict movies that the user is likely to watch and recommend the highest-ranked titles.

↓

Step 6: Personalized Movie Recommendations

Display recommended movies on the user's home page.

6. Architecture Diagram

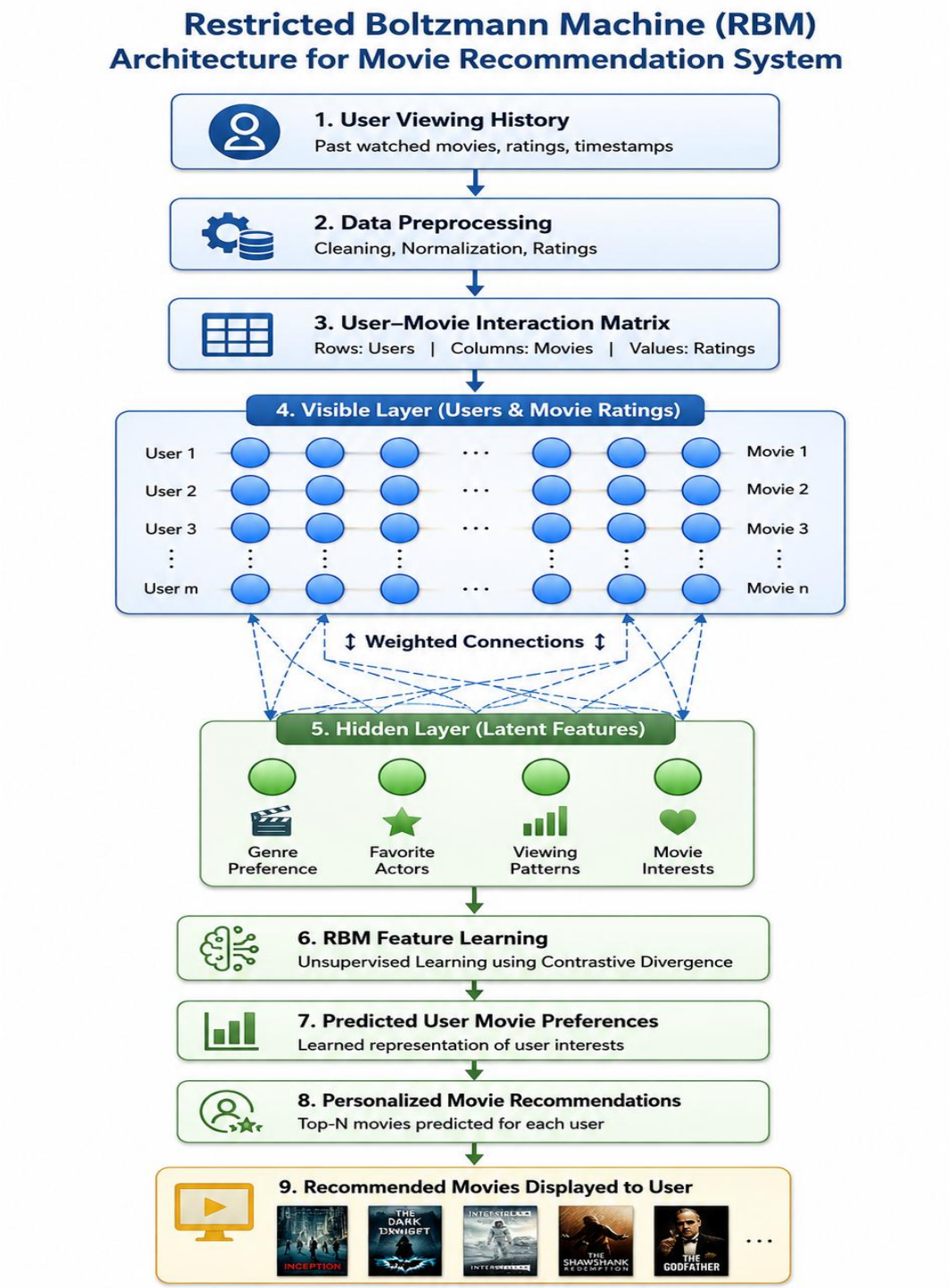


Figure 4: Restricted Boltzmann Machine (RBM) Architecture for a Movie Recommendation System

7.

7. Advantages

- Learns user preferences automatically.
- Works without labeled datasets.
- Produces personalized recommendations.
- Handles sparse rating data effectively.
- Improves user engagement.
- Scalable for large streaming platforms.

8. Limitations

- Training can be computationally intensive.
- Performance depends on the amount of user interaction data.
- Requires parameter tuning for optimal results.
- Cold-start users may receive less accurate recommendations.

9. Real-World Applications

- Movie recommendation systems
- Music recommendation platforms
- Video streaming services
- E-commerce product recommendations
- News recommendation systems

- Online content personalization

10. Conclusion

Restricted Boltzmann Machines are effective unsupervised learning models for recommendation systems. By learning hidden relationships between users and movies, RBMs can predict user preferences and provide personalized movie recommendations without requiring labeled data. This improves recommendation accuracy, enhances user experience, and increases customer engagement.

11. IEEE References

1. G. E. Hinton, "A Practical Guide to Training Restricted Boltzmann Machines," University of Toronto.

2. Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444.

3. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press.

4. R. Salakhutdinov, A. Mnih, and G. Hinton, "Restricted Boltzmann Machines for Collaborative Filtering," *Proceedings of the International Conference on Machine Learning (ICML)*.

5. Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

1. Introduction

Modern manufacturing industries continuously collect sensor data from production lines to monitor machine performance and product quality. Manual inspection is often slow, expensive, and prone to human error. A **Deep Neural Network (DNN)** combined with **Representation Learning** can automatically learn meaningful patterns from sensor data and accurately detect defective products in real time.

2. Problem Statement

The manufacturing company aims to:

- Detect defective products automatically.
- Analyze real-time sensor readings.
- Learn hidden patterns from production data.
- Improve product quality.
- Reduce manual inspection costs.
- Enable real-time quality monitoring.

3. Recommended Deep Learning Technique

Deep Neural Network (DNN) with Representation Learning

The proposed system uses a Deep Neural Network to automatically learn useful feature representations from sensor readings. Instead of manually selecting features, the network extracts important patterns through multiple hidden layers and classifies products as **Defective** or **Non-Defective**.

4. Justification

The proposed solution is effective because it:

- Automatically extracts important features from sensor data.
- Detects subtle defects that traditional methods may miss.
- Handles large volumes of real-time data efficiently.
- Improves product quality and inspection accuracy.
- Supports automated industrial quality control.
- Reduces production losses and operational costs.

5. Workflow

Step 1: Sensor Data Collection

Collect real-time sensor readings from the manufacturing process.

↓

Step 2: Data Preprocessing

- Remove noise

- Normalize sensor values
- Handle missing data
- Feature scaling

↓

Step 3: Representation Learning

The neural network automatically learns meaningful features from the sensor data.

↓

Step 4: Deep Neural Network Training

Train the model using historical production data.

↓

Step 5: Product Classification

Classify products into:

- Defective
- Non-Defective

↓

Step 6: Quality Control

Defective products are removed from the production line, while non-defective products proceed for packaging and distribution.

6.Architecture Diagram

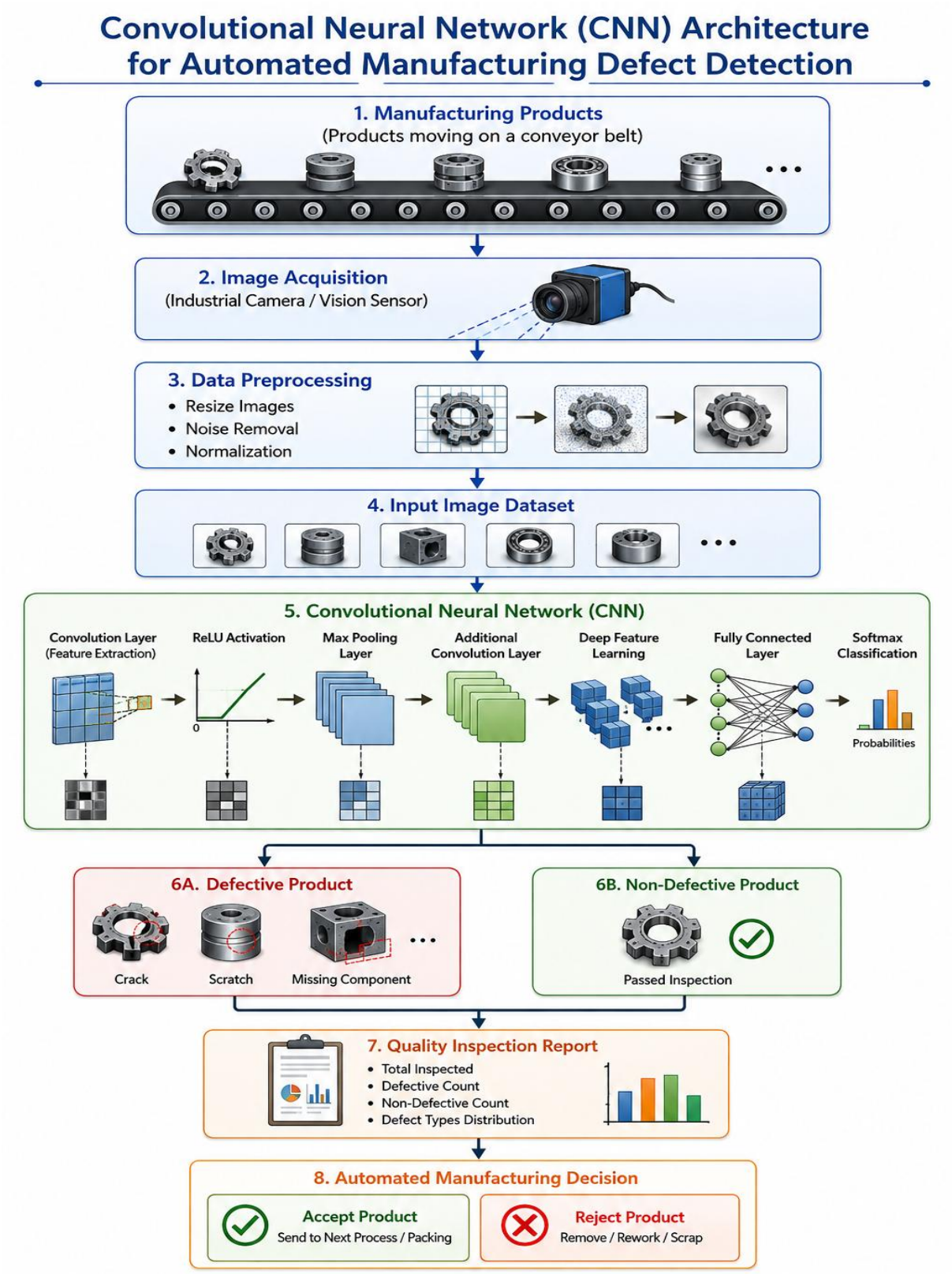


Figure 5: Convolutional Neural Network (CNN) Architecture for Automated Manufacturing Defect Detection

7. Advantages

- Automatic feature extraction.
- High defect detection accuracy.
- Real-time monitoring capability.
- Reduces manual inspection effort.
- Improves manufacturing quality.
- Scalable for Industrial IoT systems.
- Supports predictive maintenance.

8. Limitations

- Requires large amounts of training data.
- High computational requirements.
- Performance depends on sensor quality.
- Periodic model retraining may be required.

9. Real-World Applications

- Smart Manufacturing
- Industrial Quality Inspection
- Automotive Production
- Electronics Manufacturing

- Pharmaceutical Production
- Food Processing Industries

10. Conclusion

A Deep Neural Network with Representation Learning provides an effective solution for detecting defective products from real-time sensor readings. By automatically learning meaningful features and accurately classifying products as defective or non-defective, the system enhances quality control, reduces production costs, and supports intelligent manufacturing.

11. IEEE References

1. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press.
2. Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444.
3. J. Schmidhuber, "Deep Learning in Neural Networks: An Overview," *Neural Networks*, vol. 61, pp. 85–117.
4. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.